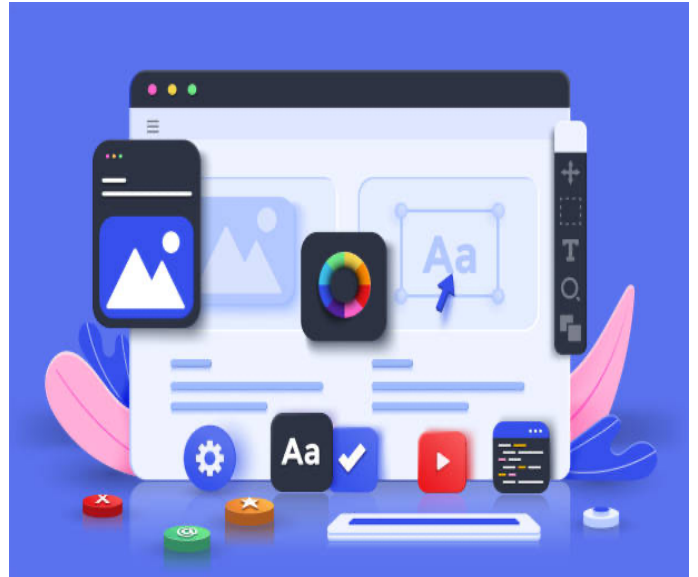


SD



Module Mijn eerste webgame

Instructie 04 – Functionaliteit Game Button programmeren

- Voorbereiden
- 'Start ronde' programmeren
- 'Reset ronde' programmeren

Auteur:	Johan Strootman
Afkorting:	SMN
Versie:	juli 2023

Inhoudsopgave

Inhoudsopgave	3
INLEIDING	5
SITUATIE 1.....	5
SITUATIE 2	5
INVENTARISATIE – SITUATIE 1	6
Inleiding.....	6
Actielijst	6
SITUATIE 1 – ONDERZOEK, VRAGEN,	7
Inleiding.....	7
De actie lijst (<i>Wat moet ik opzoeken?</i>)	8
Onderzoeksvraag 1 – Hoe bepalen we willekeurig wie mag beginnen?	10
Wat zoeken we?	10
Met welke keywords ga ik dan zoeken in Google?	10
Zoekresultaat.....	10
Resultaat voor onderzoeksvraag 1.....	13
Onderzoeksvraag 2 – Hoe starten we een ronde timer?	14
Wat zoeken we?	14
Met welke keywords ga ik dan zoeken in Google?	14
Zoekresultaat.....	14
Verklaring	15
Verder bestuderen en uitproberen	16
Resultaat van ons onderzoek	17
Oplossing voor de andere 2 onderzoeksvragen	18
Zoek eerst zelf antwoorden op onderzoeksvragen 3 en 4	18

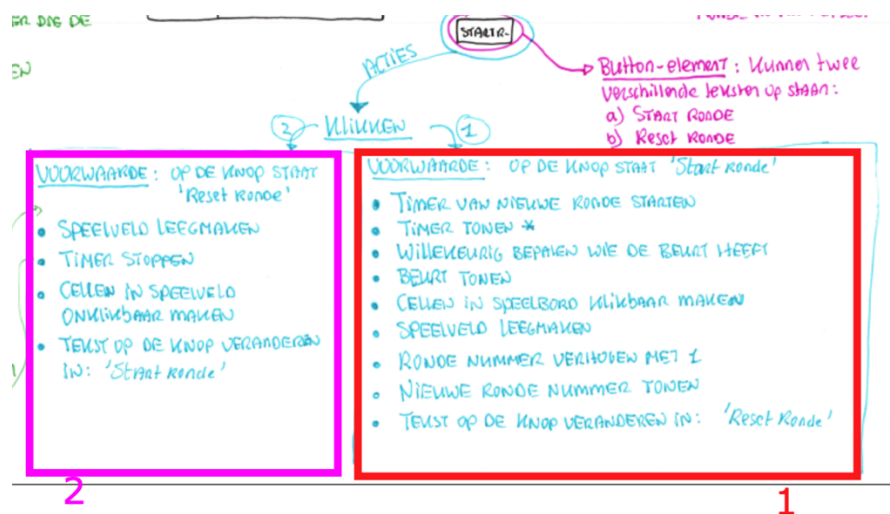
Cellen in het speelveld klikbaar maken	18
We moeten het speelveld leeg maken	19
SITUATIE 1 – FUNCTIE AFMAKEN	20
Inleiding.....	20
Resultaat	20
Toelichting op de code	21
SITUATIE 2 – SPEL RESETTEN	24
Inleiding.....	24
Resultaat	24
Toelichting.....	25
HET EINDRESULTAAT	26

INLEIDING

We gaan nu programmeren wat de button moet gaan doen wanneer een speler op de button klikt.

We hebben al in de voorbereiding in kaart gebracht wat de knop moet doen en dit was onder te verdelen in twee situaties.

De situaties zijn:



Figuur 1 - Voorwerk: Wat te doen met de button.....

SITUATIE 1

Op de knop staat de tekst '**Start ronde**'

SITUATIE 2

Op de knop staat de tekst '**Reset ronde**'

We hadden de functie daarvoor, `buttonClick(event_element)`, al aangemaakt toen we met de Setup code bezig waren, maar hadden daarbij de functie nog niet voorzien van de juiste code. Met deze functie gaan we dus nu aan de slag.

INVENTARISATIE – SITUATIE 1

Inleiding

Wat moet onze code doen wanneer de tekst '**Start ronde**' op de knop staat?

Actielijst

1. We moeten de **ronde timer starten**
2. We moeten de **rondetijd tonen**
3. Er moet (willekeurig) bepaald worden wie deze ronde mag **beginnen**
4. Er moet getoond worden wie de **beurt** heeft (nummer en symbool)
5. Cellen in het speelveld moeten **klikbaar** worden gemaakt
6. We moeten het speelveld **leeg maken**, want misschien is er nog iets te zien van een vorige ronde
7. **Ronde nummer** moet verhoogd worden
8. Nieuwe **ronde moet getoond** worden
9. De tekst op de knop moeten we veranderen in '**Reset ronde**'

SITUATIE 1 – ONDERZOEK, VRAGEN, ...

Inleiding

Wanneer we het lijstje wat we in de vorige stap hebben samengesteld langslopen zullen we ontdekken dat we bij een aantal van die actiepunten niet weten hoe we het moeten programmeren. Dit gaan we nu punt voor punt bij langs.

Het doel is dat je leert eerst te inventariseren wat je nog moet leren en hoe je daar onderzoeksvragen van maakt. En met die onderzoeksvragen via een zoekmachine op het internet op zoek gaat naar duidelijkheid, uitleg e.d.

De actie lijst (Wat moet ik opzoeken?)

1. Er moet (willekeurig) bepaald worden wie deze ronde mag **beginnen**

DIT SNAPPEN WE NIET, HOE MOET DIT? (Onderzoeksvraag 1)

HIERNA ZIE JE HOE JE HET ZOEKEN NAAR EEN OPLOSSING AANPAKT.

2. We moeten de **ronde timer starten**

DIT SNAPPEN WE NIET, HOE MOET DIT? (Onderzoeksvraag 2)

HIERNA ZIE JE HOE JE HET ZOEKEN NAAR EEN OPLOSSING AANPAKT.

3. We moeten de **rondetijd tonen**

4. Er moet getoond worden wie de **beurt** heeft (nummer en symbool)

5. Cellen in het speelveld moeten **klikbaar** worden gemaakt

DIT SNAPPEN WE NIET, HOE MOET DIT? (Onderzoeksvraag 3)

PROBEER DIT ZELF UIT TE ZOEKEN.

6. We moeten het speelveld **leeg maken**, want misschien is er nog iets te zien van een vorige ronde

DIT SNAPPEN WE NIET, HOE MOET DIT? (Onderzoeksvraag 4)

PROBEER DIT ZELF UIT TE ZOEKEN.

7. **Ronde nummer** moet verhoogd worden

8. Nieuwe **ronde moet getoond** worden

9. De tekst op de knop moeten we veranderen in '**Reset ronde**'

Na analyse van de lijst met acties die moeten worden uitgevoerd wanneer we op de button klikken zien we nu dat we 4 actiepunten hebben waarvan we niet weten hoe we dit moeten programmeren. Dit betekent voor ons dat we hier onderzoeksvragen van maken.

We gaan via Google zoeken naar oplossingen, maar dan moeten we wel weten hoe we moeten zoeken.

TIP:

Begin bij het zoeken met de term *javascript* in te tikken in de zoekbalk en pas daarna in het *Engels* aangeven wat je zoekt.

Onderzoeksvraag 1 – Hoe bepalen we willekeurig wie mag beginnen?

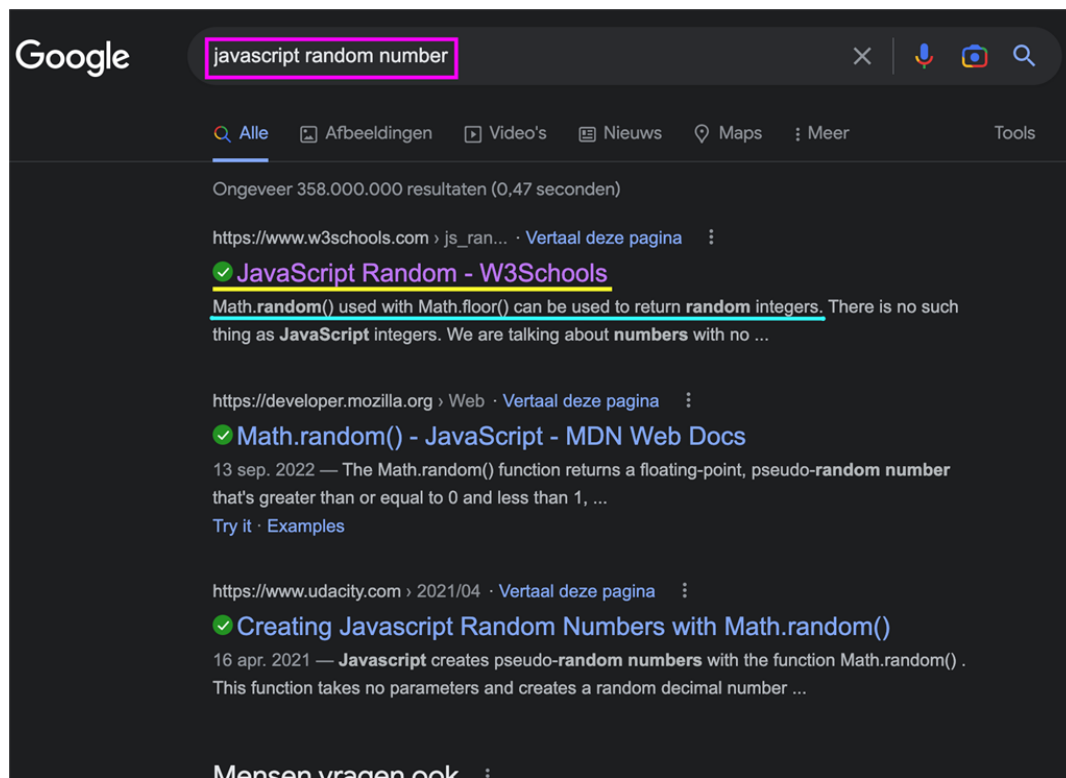
Wat zoeken we?

We zoeken een **JavaScript** methode waarmee we kunnen bepalen of speler 1 of speler 2 mag beginnen en dit moet willekeurig worden bepaald alsof er met een dobbelsteen is gegooid. We zoeken dus een JavaScript functie die **willekeurig** het **getal** 1 of 2 teruggeeft.

Met welke keywords ga ik dan zoeken in Google?

De keywords hier zijn **JAVASCRIPT**, **WILLEKEURIG** en **GETAL**. Maar **WILLEKEURIG** is niet goed, we moeten in het Engels zoeken dus bedoelen we het Engelstalige **RANDOM**. **GETAL** is ook niet goed, ook hiervoor moeten we het Engelstalige woord **NUMBER** gebruiken.

Zoekresultaat



Figuur 2 - Google Zoekresultaten

Indien nodig gaan we de verschillende links langs totdat we de link hebben gevonden die ons verder helpt. We hebben geluk, want de eerste link is meteen raak.

De eerste link levert de volgende pagina op:

[LINK: Eerste link](#)

s
es
ods
erties
s
ods
ods

JavaScript Random

< Previous

Math.random()

Math.random() returns a random number between 0 (inclusive), and 1 (exclusive):

Example

```
// Returns a random number:  
Math.random();
```

Try it Yourself »

Figuur 3 - W3Schools pagina

En als we door scrollen, ondertussen de content lezend, komen we het volgende tegen:

A Proper Random Function

As you can see from the examples above, it might be a good idea to create a proper random function to use for all random integer purposes.

This JavaScript function always returns a random number between min (included) and max (excluded):

Example

```
function getRndInteger(min, max) {  
  return Math.floor(Math.random() * (max - min) ) + min;  
}
```

Try it Yourself »

This JavaScript function always returns a random number between min and max (both included):

Example

```
function getRndInteger(min, max) {  
  return Math.floor(Math.random() * (max - min + 1) ) + min;  
}
```

Figuur 4 - Na verder scrollen op deze pagina zien we dit

En dit laatste is precies wat we bedoelen en nodig hebben. In ons geval is de waarde voor min steeds 1 en de waarde voor max steeds 2.

Na het uitgeprobeerd te hebben kunnen we tot de conclusie komen dat van het voorbeeld hierboven we eigenlijk alleen het volgende nodig hebben:

```
Math.floor(Math.random() * (max - min + 1) ) + min;
```

Figuur 5 - Het deel wat we echt nodig hebben

In de bovenstaande code vervangen we max door 2 en min door 1.

Resultaat voor onderzoeksvraag 1

We weten nu dat we de volgende constructie aan onze code moeten toevoegen om te bepalen welke speler mag beginnen. En weten ook dat we dus een globale variabele nodig hebben om het nummer dat hieruit komt in op te slaan.

Dus dit is de code die we moeten gebruiken:

```
current_player = Math.floor(Math.random() * (2 - 1 + 1) + 1);
```

Figuur 6 - De code die we in ons programma gaan gebruiken

Onderzoeksvraag 2 – Hoe starten we een ronde timer?

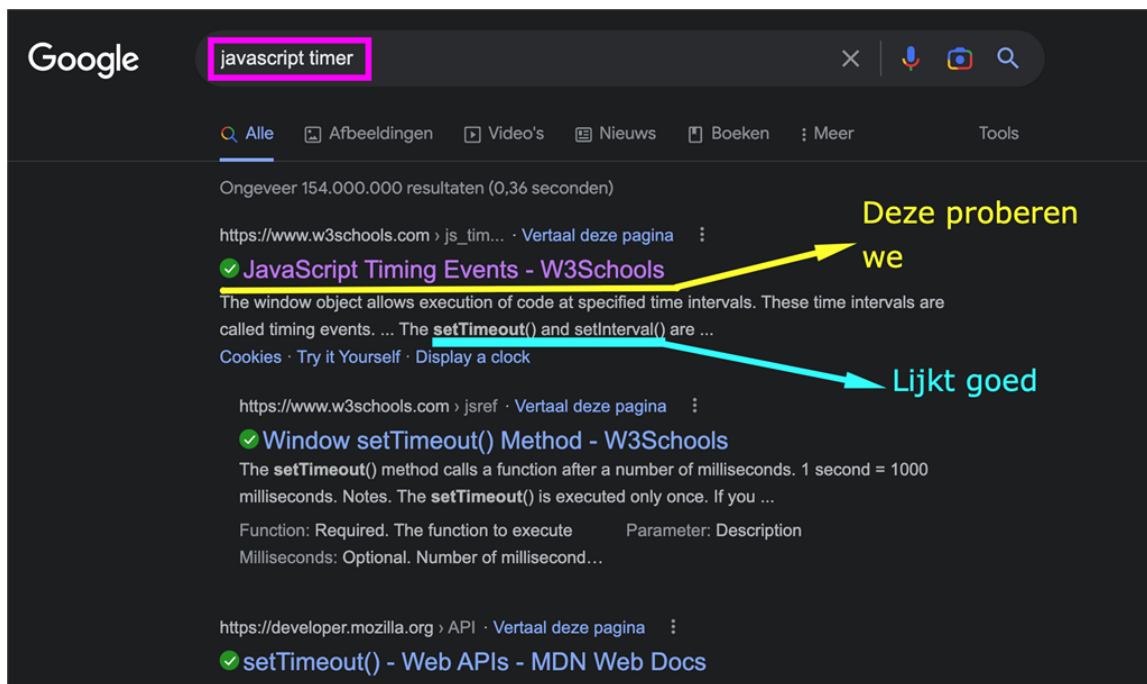
Wat zoeken we?

Een oplossing voor **JavaScript** en het gaat om het starten van een lopende **timer**.

Met welke keywords ga ik dan zoeken in Google?

De keywords hier zijn **JAVASCRIPT** en **TIMER**.

Zoekresultaat

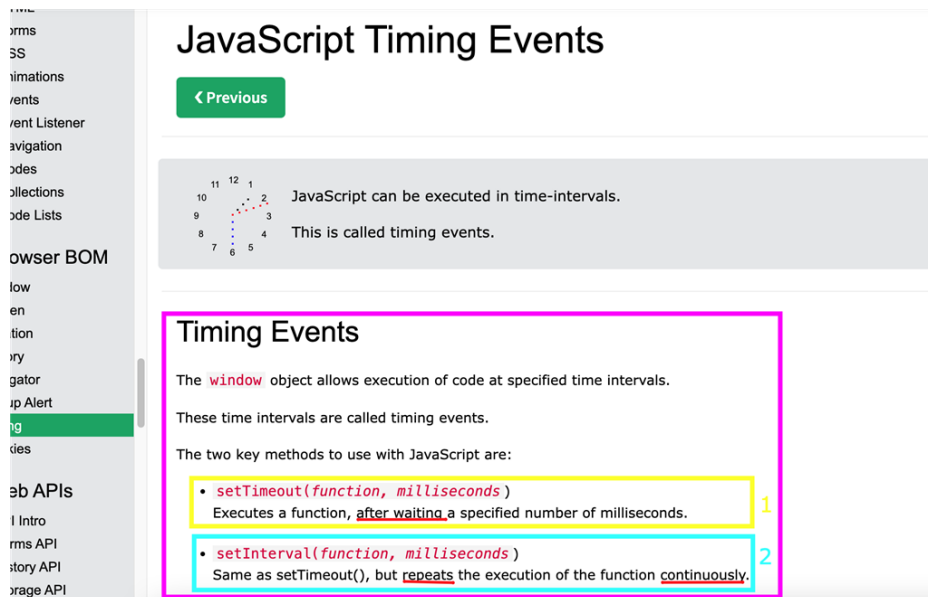


Figuur 7 - Google zoekresultaten

We gaan net zo lang langs de zoekresultaten tot we antwoorden vinden.

We beginnen met de eerste link en komen dan op onderstaande pagina:

[LINK: Eerste gevonden link](#)



Figuur 8 - W3Schools pagina

Verklaring

1. Dit lijkt niet goed, we lezen de tekst '*after waiting*' oftewel de functie die wordt pas uitgevoerd na een x aantal milliseconden te hebben gewacht. Dit is niet wat wij willen.

DIT IS DUS NIET DE MANIER!

2. Dit lijkt goed, want we lezen hier dat een functie continue om de zoveel milliseconden wordt uitgevoerd. Dit is precies wat wij willen.

DIT IS DUS DE MANIER!

Dit betekent dat we hier verder in duiken om te ontdekken hoe we het dan in ons geval kunnen gebruiken.

Verder bestuderen en uitproberen

Als we op de gevonden pagina verder gaan scrollen zien we het volgende:

The setInterval() Method

The `setInterval()` method repeats a given function at every given time-interval.

`window.setInterval(function, milliseconds);`

Uitleg over hoe we deze functie moeten gebruiken

The `window.setInterval()` method can be written without the window prefix.

The first parameter is the function to be executed.

The second parameter indicates the length of the time-interval between each execution.

This example executes a function called "myTimer" once every second (like a digital watch).

Example

Display the current time:

```
setInterval(myTimer, 1000);

function myTimer() {
  const d = new Date();
  document.getElementById("demo").innerHTML = d.toLocaleTimeString();
}
```

Voorbeeld

Try it Yourself »

Uitproberen

Figuur 9 - Na verder scrollen op de pagina ontdekken we dit

EN

How to Stop the Execution?

The `clearInterval()` method stops the executions of the function specified in the `setInterval()` method.

`window.clearInterval(timerVariable)`

The `window.clearInterval()` method can be written without the window prefix.

The `clearInterval()` method uses the variable returned from `setInterval()` :

```
let myVar = setInterval(function, milliseconds);
clearInterval(myVar);
```

Figuur 10 - Nog een stukje verder scrollen en we ontdekken hoe we een timer kunnen stoppen

Resultaat van ons onderzoek

We weten nu, nadat we dit hebben gelezen en eventueel hebben uitgetoetst, dat we deze oplossing dus kunnen gebruiken in ons eigen spel.

We weten namelijk door de voorbereidingen dat we een timer moeten kunnen starten, maar dat we deze ook weer moeten kunnen stoppen. Beide oplossingen hebben we op deze webpagina gevonden.

Dit betekent voor onze code het volgende:

- We hebben nog een **globale variabele** nodig waarin we de ID van de gestarte timer kunnen bewaren, want alleen met de ID kunnen we deze zelfde timer later weer stoppen.
- We weten nu dat om een timer te starten we de JavaScript functie **setInterval** kunnen gebruiken
- We weten dat de JavaScript functie **setInterval** vraagt om **twee parameters**. De eerste is de naam van de zelf te schrijven functie die steeds uitgevoerd moet worden en de tweede waarmee we aangeven om de hoeveel milliseconden de functie uitgevoerd moet worden.
- We weten nu dat we de JavaScript functie **clearInterval** kunnen gebruiken om een al gestarte timer te stoppen en dat deze als parameter een ID van de gestarte timer moet krijgen van ons.

Oplossing voor de andere 2 onderzoeksvragen

Zoek eerst zelf antwoorden op onderzoeksvragen 3 en 4

Uiteraard ga je eerst zelf op zoek naar oplossingen en antwoorden, want waar het om draait is dat je weet hoe je moet zoeken, hoe je gevonden oplossingen aanpast zodat het werkt in jouw applicatie. Dit is een belangrijke vaardigheid voor developers namelijk.

Cellen in het speelveld klikbaar maken

De cellen in het speelveld hebben we met de onderstaande code in onze Setup binnengehaald:

```
element_cells = document.querySelectorAll('.gameboard img');
```

Figuur 11 - Binnenhalen van alle cellen in JavaScript

Dit levert ons in de variabele `element_cells` een array op met alle 9 de IMG-tags van het speelveld:



Figuur 12 - In de console van de Inspector bekijken we de variabele `element_cells`

Nu moeten we nog langs alle 9 IMG-tags lopen om ervoor te zorgen dat spelers erop kunnen klikken. Als je het onderzoek goed hebt gedaan heb je ook kunnen vinden dat een array in feite een object is met niet alleen properties (eigenschappen) maar ook methods (functionaliteit). En een van de methods die we in een array tot onze beschikking hebben is `forEach` (zie [LINK: W3Schools - Array's en forEach](#)).

We zijn dan uiteindelijk tot de conclusie gekomen dat we het klikbaar maken van de cellen op de volgende manier kunnen programmeren:

```
element_cells.forEach(cell => {  
  cell.addEventListener('click', cellClick);  
});
```

Figuur 13 - Click Event Listener toevoegen aan elke cel

Hierin is `cellClick` een zelfgeschreven functie die we nog wel moeten aanmaken in onze code. En deze functie geven we dan als volgt vorm:

```
function cellClick(event_element)  
{  
    
}
```

Figuur 14 - Cell Click EventListener

We moeten het speelveld leeg maken

In feite hebben we in het vorige onderzoek al geleerd hoe we langs al die cellen kunnen lopen, namelijk met de `forEach` method in een array. Dezelfde method gebruiken we nu om de **SRC-attribuut** van de images te veranderen:

```
element_cells.forEach(cell => {  
  cell.src = 'img/empty.jpg';  
});
```

Figuur 15 - Iedere cell bij start vullen met een lege image

SITUATIE 1 – FUNCTIE AFMAKEN

Inleiding

We zijn alleen nog maar bezig met het afmaken van het deel van de `buttonClick` functie waarin we dingen gaan doen wanneer de tekst **'Start ronde'** op de knop staat. Dat deel van de functie gaan we nu afmaken en daarbij coderen we iedere actie uit de actielijst.

Resultaat

```
83     if(event_element.target.innerHTML == 'Start ronde') {
84
85         current_round++; // Ronde nummer met 1 verhogen
86         element_rounds_played.innerHTML = current_round; // Nu plaatsen we het nummer in het element
87         // om het te tonen aan de spelers
88
89         // Willekeurig bepalen welke speler mag beginnen, dit levert een 1(speler 1) of een 2(speler 2) op
90         current_player = Math.floor(Math.random() * (2 - 1 + 1) + 1);
91
92         // Nu gaan we tonen op het scherm welke speler de ronde mag beginnen
93         element_turn_number.innerHTML = current_player; // Het nummer tonen
94         element_turn_image.src = (current_player == 1 ? 'img/circle200x200.png' : 'img/cross200x200.png');
95
96         /*
97          * In de onderstaande programma lus lopen we nu langs alle cellen in het speelbord
98          * en koppelen hier een click eventhandler aan.
99          */
100        element_cells.forEach(cell => {
101            cell.addEventListener('click', cellClick);
102        });
103
104        event_element.target.innerHTML = 'Reset ronde'; // Tekst op de knop veranderen
105        element_round_time.innerHTML = '00:00'; // We resetten en tonen een ronde tijd 00:00
106
107        /*
108         * We starten nu de ronde timer
109         * Als echter de variabele timer_id niet gelijk is aan NULL dan loopt er nog
110         * een timer, deze moeten we eerst stoppen.
111         */
112        if(timer_id != null) { // Als de variabele timer_id niet NULL is is er al een timer
113            clearInterval(timer_id); // Stop de timer
114            timer_id = null; // Reset de variabele op NULL
115            elapsedTimeInSeconds = 0; // Reset de variabele op 0 seconden
116        }
117
118        timer_id = setInterval( roundsTimer, 1000 ); // Om de seconde mag de functie roundsTimer uitgevoerd worden
119    } else {
```

Figuur 16 - Dit moet de code worden voor 'Start ronde'

Toelichting op de code

Regel 83

Op deze regel controleren we eerst met een **IF**-statement of de tekst 'Start ronde' op de knop staat. De tekst van een element staat dan in een property met de naam **innerHTML** of **innerText**. Eén van deze properties kunnen we dan gebruiken om de inhoud hiervan te vergelijken met de tekst 'Start ronde'.

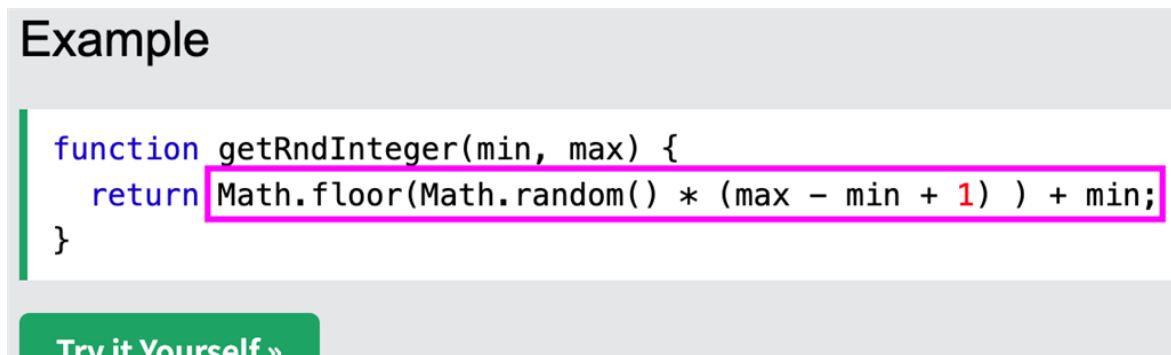
Regel 85 & 86

Op regel 85 verhogen we alvast het ronde nummer en op regel 86 plaatsen we deze in de **innerHTML** property van het HTML-element *element_rounds_played*. Door het hier in de property te plaatsen zien we het ook direct terug op het scherm.

Regel 90

Dit is dan de constructie om de computer willekeurig een 1 of een 2 te laten genereren. Dit is bedoeld om vast te stellen welke speler mag beginnen. Er is echter een belangrijk nadeel aan de manier waarop we dit gecodeerd hebben.

We vonden tijdens ons onderzoek de volgende oplossing op W3Schools:



Figuur 17 - De code in de roze rechthoek vonden we toepasbaar in onze code

En deze oplossing hebben we iets aangepast zodat het past in onze code. Zo hebben we **max** vervangen door **2**, wat staat voor speler 2 en **min** vervangen door **1**, wat weer staat voor speler 1. In de oplossing stond ook al een waarde 1, maar de betekenis van deze waarde staat niet gelijk aan een nummer voor een speler.

Het resultaat is dus:

```
current_player = Math.floor(Math.random() * 2 - 1 + 1) + 1;
```

Figuur 18 - Resultaat na de nodige aanpassingen

Maar dit is eigenlijk onleesbaar, want als we het later teruglezen of een andere programmeur moet met onze code aan de slag dan rijst de vraag: 'Waar staan de getallen voor? Welke waarde 1 heeft welke betekenis?'

Dit is iets om te onthouden, want dit gaan we later aanpassen in een **REFACTORING fase**.

Regel 93

Op deze regel tonen we nu het nummer van de speler die mag beginnen.

Regel 94

Op deze regel tonen we nu het symbool van de speler die mag beginnen.

Maar wat zien we hier voor eigenaardige constructie in de code?

```
element_turn_image.src = (current_player == 1 ? 'img/circle200x200.png' : 'img/cross200x200.png');
```

Figuur 19 - Eigenaardige code: Ternary Operator

Deze constructie noemen we een **Ternary Operator**.

Dit is een sterk **verkorte versie van een if.....else.... statement**.

Als we bovenstaande regel zouden schrijven met als een if.....else.... statement zou deze er als volgt uitzien:

```
if (current_player == 1) {  
    element_turn_image.src = 'img/circle200x200.png';  
} else {  
    element_turn_image.src = 'img/cross200x200.png';  
}
```

Figuur 20 - De ternary operator vertaald naar een IF-ELSE statement

Maar zoiets simpels als het toekennen van een nieuwe waarde aan een variabele op basis van een controle (voorwaarde) maken we liever gebruik van de **Ternary Operator**. Deze is veel korter. Zie [LINK: W3Schools - Ternary Operator](#) voor meer uitleg over de Ternary Operator.

Regel 100 t/m 102

Hier kennen we aan iedere cell in het speelveld een **click eventhandler** toe.

Regel 104 en 105

Op regel 104 veranderen we de tekst op de knop in 'Reset ronde' en op regel 105 laten we de starttijd voor de ronde, namelijk 00:00, zien op het scherm.

Regel 112 t/m 116

Op regel 112 controleren we met een **IF**-statement eerst of er al een timer loopt, want als er al eentje loopt is de waarde in de variabele niet gelijk aan **NULL** namelijk. Zo hebben we dat zelf geprogrammeerd toch?

Als de waarde in de variabele **niet NULL** is dan loopt er dus een timer en moeten we deze eerst stoppen voordat we een nieuwe timer starten. Op regel 113 voeren we opdracht uit om de timer te stoppen.

Op regel 114 en 115 resetten we de hulpvariabelen. Op regel 114 zetten we waarde van de variabele, waarin we de ID van een timer opslaan, op **NULL**. En op regel 115 zetten we de waarde van de variabele waarin we iedere verlopen seconde in de timer opslaan, op 0.

Regel 118

Hier maken we een nieuwe timer aan en starten deze.

Regel 119

Hier zie je alvast het **ELSE** gedeelte van de **IF**-statement staan, want in het ELSE gedeelte gaan we straks de code plaatsen die moet worden uitgevoerd als op de knop de tekst '**Reset ronde**' bevat.

Maar waarom hier dan geen nieuwe IF-statement?

Dit is niet nodig. Omdat we zelf programmeren en dus zelf bepalen wanneer welke tekst op de knop staat, weten we dat als er geen 'Start ronde' op staat het dan wel 'Reset ronde' moet zijn. Andere teksten laten we ons programma niet op de knop plaatsen toch? Dus kunnen we hier volstaan met een **ELSE** gedeelte.

SITUATIE 2 – SPEL RESETTEN

Inleiding

Het deel van de functie wat moet reageren op een klik op de knop wanneer de tekst '**Start ronde**' op de knop staat is nu klaar.

Maar we hebben nog niet geprogrammeerd wat de functie moet doen wanneer de tekst '**Reset ronde**' op de knop staat.

Dit gaan we nu doen. Daarvoor gebruiken we uiteraard ook weer onze aantekeningen en schetsen uit het voorbereidend werk.

Resultaat

```
118     timer_id = setInterval( roundsTimer, 1000 ); // Om de seconde maag de functie roundsTimer uitgevoerd worden
119 } else {
120     /* Nee, dan gaan we het spel resetten door de volgende
121     * stappen uit te voeren:
122     * - Timer stoppen
123     * - Laatste timer info tonen op het scherm
124     * - Speelveld leeg maken (overal weer empty.jpg in plaatsen) en
125     *   Deactiveren van de click event handlers op de cellen
126     * - Start spel op de button plaatsen
127     */
128
129     /*
130     * We gaan nu eerst alle cellen in het speelveld onklikbaar maken
131     */
132     element_cells.forEach(cell => {
133         cell.removeEventListener('click', cellClick);
134         cell.src = 'img/empty.jpg';
135     });
136
137     event_element.target.innerHTML = 'Start ronde'; // Tekst op de knop veranderen
138
139     // Nu stoppen we de ronde timer
140     if(timer_id != null) { // Als de variabele timer_id niet NULL is is er al een timer
141         clearInterval(timer_id); // Stop de timer
142         timer_id = null; // Reset de variabele op NULL
143         elapsedTimeInSeconds = 0; // Reset de variabele op 0 seconden
144     }
145 }
146 }
```

Figuur 21 - 'Reset ronde' deel van de knop functie

Dit deel van de functie is het simpelst en gaan we regel voor regel hieronder toelichten.

Toelichting

Regel 132 t/m 135

We lopen met de method `forEach` langs alle cellen in de array `element_cells` en op regel 133 deactiveren we de click eventhandler van iedere cell. Dit kunnen we doen door gebruik te maken van de standaard aanwezige method `removeEventListener`. Op regel 134 plaatsen we in iedere cel het afbeeldingsbestand `empty.jpg` in de **SRC-attribuut**. Hiermee maken we het speelveld leeg.

Regel 137

We veranderen de tekst van de knop weer in 'Start ronde'.

Regel 140 t/m 144

Hier stoppen we weer de timer (als deze bestaat) en resetten de daarbij behorende hulpvariabelen.

HET EINDRESULTAAT

Uiteindelijk moet de knop functie er als volgt uit gaan zien:

```
76 function buttonClick(event_element)
77 {
78     /*
79     We controleren hier of de tekst op de button gelijk is aan: 'Start ronde'
80     Want als dat zo is dan moet er namelijk iets anders gebeuren dan wanneer
81     de tekst 'Reset ronde' op de button staat.
82     */
83     if(event_element.target.innerHTML === 'Start ronde') {
84         current_round++; // Ronde nummer met 1 verhogen
85         element_rounds_played.innerHTML = current_round; // Nu plaatsen we het nummer in het element
86         // om het te tonen aan de spelers
87
88         // Willekeurig bepalen welke speler mag beginnen, dit levert een 1(speler 1) of een 2(speler 2) op
89         current_player = Math.floor(Math.random() * (2 - 1 + 1) + 1);
90
91         // Nu gaan we tonen op het scherm welke speler de ronde mag beginnen
92         element_turn_number.innerHTML = current_player; // Het nummer tonen
93         element_turn_image.src = (current_player == 1 ? 'img/circle200x200.png' : 'img/cross200x200.png');
94
95         /*
96         In de onderstaande programma lus lopen we nu langs alle cellen in het speelbord
97         en koppelen hier een click eventhandler aan.
98         */
99         element_cells.forEach(cell => {
100             cell.addEventListener('click', cellClick);
101         });
102
103         event_element.target.innerHTML = 'Reset ronde'; // Tekst op de knop veranderen
104         element_round_time.innerHTML = '00:00'; // We resetten en tonen een ronde tijd 00:00
105
106         /*
107         * We starten nu de ronde timer
108         * Als echter de variabele timer_id niet gelijk is aan NULL dan loopt er nog
109         * een timer, deze moeten we eerst stoppen.
110         */
111         if(timer_id != null) { // Als de variabele timer_id niet NULL is is er al een timer
112             clearInterval(timer_id); // Stop de timer
113             timer_id = null; // Reset de variabele op NULL
114             elapsedTimeInSeconds = 0; // Reset de variabele op 0 seconden
115         }
116
117         timer_id = setInterval( roundsTimer, 1000 ); // Om de seconde mag de functie roundsTimer uitgevoerd worden
118     } else {
119         /* Nee, dan gaan we het spel resetten door de volgende
120         * stappen uit te voeren:
121         * - Timer stoppen
122         * - Laatste timer info tonen op het scherm
123         * - Speelveld leeg maken (overall weer empty.jpg in plaatsen) en
124         * Deactiveren van de click event handlers op de cellen
125         * - Start spel op de button plaatsen
126         */
127
128         /*
129         * We gaan nu eerst alle cellen in het speelveld onklikbaar maken
130         */
131         element_cells.forEach(cell => {
132             cell.removeEventListener('click', cellClick);
133             cell.src = 'img/empty.jpg';
134         });
135
136         event_element.target.innerHTML = 'Start ronde'; // Tekst op de knop veranderen
137
138         // Nu stoppen we de ronde timer
139         if(timer_id != null) { // Als de variabele timer_id niet NULL is is er al een timer
140             clearInterval(timer_id); // Stop de timer
141             timer_id = null; // Reset de variabele op NULL
142             elapsedTimeInSeconds = 0; // Reset de variabele op 0 seconden
143         }
144     }
145 }
146
147
```

Figuur 22 - Dit is wat we uiteindelijk geprogrammeerd hebben

